



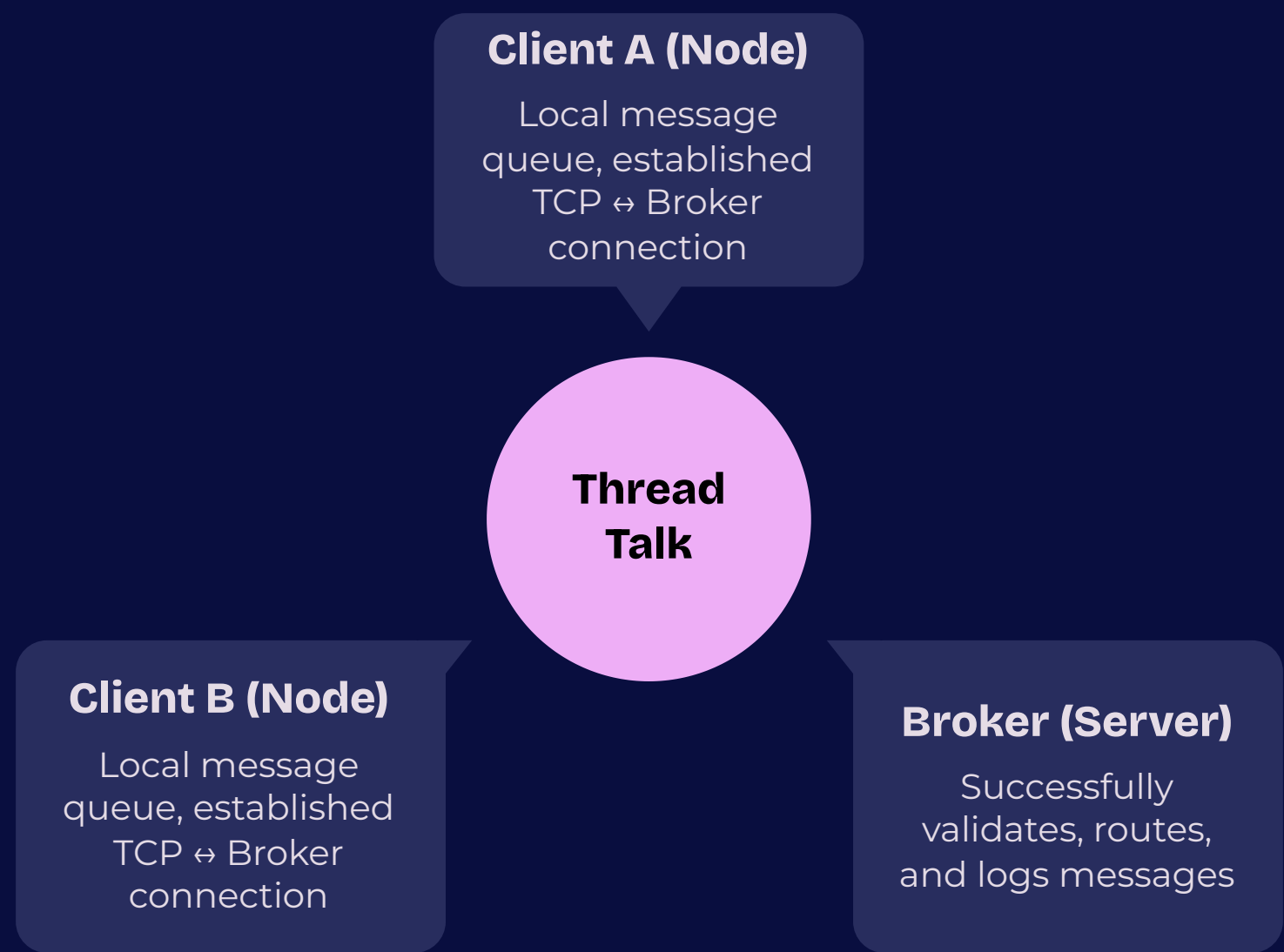
# Thread Talk: A Broker-Based Distributed Chat Application

Successfully developed and fully implemented: A robust distributed chat system that enables multiple clients across networked machines to communicate seamlessly in real-time via a central broker.

📄 **Jadah Holly, Temidayo Akingemi and Eleni Mekonnen**

November 20, 2025

# Thread Talk High-Level Architecture - Fully Implemented



## Broker (Server) - Fully Functional

The central controller has been successfully implemented and is now responsible for managing client connections, message validation, delivery routing, and message logging.



## Clients (Nodes) - Operational

Independent processes running on networked machines have been deployed. Each process effectively utilizes internal threads for concurrent input/output handling.



## Inter-Process Communication (IPC) - Established

Local message buffering within each client process is robustly handled using implemented OS-level constructs like message queues or pipes, ensuring seamless communication.

# Key Features & Implementation



## Broker-Centric Routing

Reliable message routing through a central broker, ensuring all communications are managed and delivered efficiently.



## Concurrent Client Operations

Support for multiple clients with multithreading, allowing for simultaneous input/output handling and a responsive user experience.



## Network Communication

Robust TCP/IP distributed communication enables seamless interaction across different networked machines.



## Broker Module

This module accepts TCP connections, maintains an active client registry, efficiently relays messages between clients, and implements comprehensive logging for all activities.



## Client Module

Responsible for connecting to the broker, spawning dedicated sender/receiver threads, utilizing local Inter-Process Communication (IPC), and handling all user input/output operations.

# Code Implementation: Fully Implemented Components

```
server.py x client.py json_log.py broker_log.py
server.py > ...
16 def handle_clients(connection, address):
17     log_event('connection', username)
18     print(f"[REGISTRATION] {username} has joined")
19
20     while True:
21         data = connection.recv(1024)
22         if not data:
23             break
24
25         try:
26             packet = json.loads(data.decode())
27         except json.JSONDecodeError:
28             print("[ERROR] Invalid packet received")
29             continue
30
31         route_message(connection, packet)
32
33     except Exception as e:
34         print(f"[ERROR] {e}")
35
36     finally:
37         # clean the connection and remove the registry
38         if connection in clients:
39             username = clients[connection]
40             del clients[connection]
41             del user_sockets[username]
42             connection.close()
43
44
45
46
47
48
49
50
51
```

## server.py - Broker Module

The **server.py** component has been fully implemented, successfully managing and accepting multiple client connections, identifying users, routing packets based on "type" - "chat", "direct message", "subscription", and "publish"

```
def start_client():
    # create tcp/ip socket
    client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

    # connect to the broker (central server)
    # replace the 'localhost' and 3802 with the broker IP and port if running remotely
    client.connect(('localhost', 3802))
    print("[CONNECTED] Connected to the broker at localhost:3802")

    # identify the client with a username (authentication)
    username = input("Enter username. \n>> ")
    client.send(json.dumps({
        "type": "register",
        "username": username,
    }).encode()) # send the username to the broker as JSON

    # generate two threads
    # receiver thread to run receive() to consistently print incoming messages
    # sender thread to run send() to allow simultaneous user input
    receiver_thread = threading.Thread(target=receive, args=(client,))
    sender_thread = threading.Thread(target=send, args=(client,))

    # start both threads
    receiver_thread.start()
    sender_thread.start()

    # run the thread indefinitely until the user disconnects or the broker (server) just stops
    receiver_thread.join()
```

## client.py - Client Module

The **client.py** module is now fully functional, effectively utilizing two internal threads: a sender thread for handling user input and dispatching messages to the broker, and a receiver thread for listening to and displaying all incoming messages, ensuring a seamless chat experience.

# Output & Demonstration



Improved Code – Parallel and Distributed Programming.mp4



# GUI Output



Improved Code – GUI.mp4



# ThreadTalk: Project Completed & Final Results

The core infrastructure is fully functional and demonstrated, achieving the following key milestones as part of the completed ThreadTalk distributed chat application:

## Multi-Client Server Fully Implemented

The `server.py` now successfully accepts and manages multiple client connections, including robust username mapping for identification.

## Seamless Concurrent Client Operations

The `client.py` process utilizes dedicated send and receive threads, enabling robust, fully concurrent message handling across all connected users.

## End-to-End Communication Validated

A comprehensive working demonstration has validated complete, reliable, and concurrent message transmission between all clients via the server, proving the application's stability.

Final Enhancements: Error handling has been comprehensively enhanced, message persistence successfully implemented, and scaling options explored, resulting in a production-ready system.

# Mapping Requirements to Implementation Status - Thread Talk Application

| Requirement                | Status | Description                                                                                                                                           |
|----------------------------|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| Multiprocessing            | ● Done | Multiple clients are instantiated as separate OS processes, allowing for true distribution.                                                           |
| Multithreading             | ● Done | Dedicated send and receive threads are correctly allocated per client process to handle concurrent I/O.                                               |
| Interprocess Communication | ● Done | Robust message queues (IPC) have been fully implemented for reliable local buffering between client threads.                                          |
| Distributed Communication  | ● Done | TCP sockets are fully initialized and tested for seamless communication between the Broker and Clients.                                               |
| Communication Broker       | ● Done | The Communication Broker is fully functional, accepting connections, identifying clients, and relaying messages with high reliability and efficiency. |

All core requirements for the Thread Talk distributed chat application have been successfully implemented and validated, marking the project's complete finish and readiness.



# Summary and Looking Ahead: Key Takeaways

## Core System Fully Implemented

The fundamental distributed architecture (Broker + Concurrent Clients) is fully functional, successfully demonstrated, and completely implemented in a command-line environment.



## Robustness & Reliability Achieved

Comprehensive error handling and logging improvements have been fully implemented, significantly bolstering system reliability and maintainability.

## Advanced Features Delivered

Advanced scalability features and secure client authentication have been successfully implemented, fully preparing Thread Talk with expanded functionality.

Thread Talk has successfully delivered a fully functional, distributed chat simulator, effectively demonstrating key concepts of networked systems. Its robustness and scalability have been significantly enhanced and are now complete.